

< Firework를 응용한 R프로그래밍 >

과 목	R 프로그래밍
담당 교수	정재환
제출 일자	2024. 12. 23.
전공	정보통계학과
학번	2019068067
이름	유윤종

- 목 차 -

I. 서론

II. 본론

1. Firework

2. 오토배틀러 시뮬레이션

I. 서론

본 보고서는 R 그래픽을 이용한 두가지 시각화를 다룰 것이다. 먼저 Firework 관련 기존 코드에 관하여 함수별로 구조와 구현 원리를 살펴볼 것이다. Firework에서는 Unif_on_Circle() 함수와 Fireworks() 함수가 어떻게 작동되는지에 대해 탐구할 것이다. 그 다음으로는 오토배틀러 시뮬레이션에 대해 살펴볼 것이다. 이 게임은 오토배틀러를 시뮬레이션한 뒤 여러가지 이펙트를 Firework로 구현한 게임 형태의 시각화 코드이다.

II. 본론

1. Firework

(1) 주요 옵션 정리

옵션	정의	예시
runif()	지정된 범위 내에서 균등 분포를 따르는 난수를 생성	runif(n, min, max)
sample()	주어진 벡터에서 임의로 샘플을 추출하거나 순서를 섞음	sample(1:10, 5, replace=T)
points()	기존의 플롯에 점(point)을 추가하여 그래프를 확장	points(x, y, col = "red")
lines()	기존의 플롯에 선(line)을 추가하여 그래프를 확장	lines(x, y, type = "l")

(2) Unif_on_Circle 함수 정의

```
Unif_on_Circle = function(a, b, r, n)
{
  # x는 균등분포
  x = runif(n, a - r, a + r)
  # y는 x가 주어졌을 때 조건부분포가 균등분포

  y = NULL
  for(i in 1 : n)
    y[i] = runif(1, - sqrt(r^2 - (x[i] - a)^2) + b,
                sqrt(r^2 - (x[i] - a)^2) + b)
  return(cbind(x, y))
}
```

Unif_on_Circle(a, b, r, n) 함수는 중심이 (a,b)이고 반지름이 r인 원 내부에 균등하게 분포된 n개의 점을 생성하는 역할을 수행한다. 구현방식은 다음과 같다. 먼저 x 좌표를 [a-r,a+r] 구간에서 균

등분포로 n 개 생성하고, 각 $x[i]$ 에 대해 해당하는 원 내부의 범위인 $y = b - \sqrt{r^2 - (x - a)^2} \sim b + \sqrt{r^2 - (x - a)^2}$ 에서 무작위로 하나의 점을 추출한다. 이렇게 구해진 x 와 y 좌표를 `cbind()`를 이용하여 쌍으로 묶는다.

(3) Fireworks 함수 정의 및 주요 단계 설명

```
Fireworks = function(num_fire, num_points, sleep = 0.1)
```

Fireworks 함수를 정의하였다. 이 함수의 옵션은 다음과 같다. `num_fire`는 폭죽의 개수를 결정한다 (default = 10). 그 다음 `num_points`는 하나의 폭죽에서 그려지는 점의 개수를 지정한다. 마지막으로 `sleep` 옵션은 움직임을 위한 `Sys.sleep`에 대응되는 옵션이다.

```
center_a = runif(num_fire, -1, 1)
center_b = runif(num_fire, -1, 1)
center_ab = cbind(center_a, center_b)
r_vec = runif(num_fire, 0.1, 0.3)
```

폭죽이 생성될 위치를 $[-1, 1]$ 범위 내에서 무작위로 선택하고, 그 폭죽에 대해 각각 반지름을 0.1에서 0.3 사이의 난수로 설정한다

```
fire_points_list = list()
for(i in 1:num_fire)
  fire_points_list[[i]] = Unif_on_Circle(center_ab[i, 1], center_ab[i, 2],
                                         r_vec[i], n = num_points)
```

각 폭죽에서 추출한 점들의 좌표를 저장할 리스트를 먼저 초기화하고 for 반복문을 돌면서 각 폭죽을 처리하는 역할을 수행한다. 이때 `Unif_on_Circle` 함수를 이용하여 원 내부에서 무작위로 `num_points`개의 점을 뽑고, `fire_points_list`에 저장한다.

```
par(pty = "s", bg = "black")
plot(0, 0, xlim = c(-1, 1), ylim = c(-1, 1), type = "n", bty = "n",
     axes = F, xlab = "", ylab = "")
rain_col = sample(rainbow(num_points, alpha = 0.3))
```

기본 그래픽 옵션을 설정하는 부분이다. 먼저 불꽃놀이 시각화가 돋보이기 위해 배경을 검은색으로 설정하고, `plot` 함수를 이용하여 그래프 영역을 생성하나 어떠한 것 하나 그리지 않고 좌표 영역만 확보한다. 그리고 `sample`, `rainbow` 함수를 사용하여 무지개색 팔레트를 생성해 불꽃놀이의 색깔을 다양화한다.

```
for (i in 1 : num_fire)
{
  for (j in 1 : num_points)
```

```

        lines(x = c(center_ab[i, 1], fire_points_list[[i]][j, 1]),
              y = c(center_ab[i, 2], fire_points_list[[i]][j, 2]),
              col = rain_col[j], lwd = 1)
# 연쇄적인 시각화를 위한 작업
if (i > 2)
  points(fire_points_list[[i - 2]],
         col = rain_col, cex = 0.5)
Sys.sleep(sleep)
}
# 나머지 2개
for(i in c(num_fire - 1, num_fire))
{
  points(fire_points_list[[i]],
         col = rain_col, cex = 0.5)
  Sys.sleep(sleep)
}
# 마지막 점
plot(0, 0, xlim = c(-1, 1), ylim = c(-1, 1), type = "n", bty = "n",
     axes = F, xlab = "", ylab = "")
for(i in 1 : num_fire)
{
  points(fire_points_list[[i]],
         col = rain_col, cex = 0.5)
}

```

각 폭죽의 중심에서 폭죽을 이루는 점들까지 선을 그리는 작업을 수행한다. 한 폭죽에 대해, 샘플링된 점이 num_points개 있으므로, 각 점에 대해 반복문을 돌면서 중심과 점을 연결하는 선을 그린다. 이미 그렸던 폭죽이 사라지지 않도록, 혹은 화면에 잔상이 남도록 일부 점들을 다시 생성한다. 첫번째 for문이 모두 돌았다면 마지막 두개의 폭죽을 그리기 위해 점을 다시 생성한다.

2. 오토배틀러 게임 시뮬레이션

오토배틀러는 게임 장르의 한 종류로써 플레이어가 직접 개입하지 않고, 여러가지 기물들을 구매하고 조합하여 팀을 구성한 뒤 상대가 구성한 팀과 '대신' 싸워주는 장르로써 많은 인기를 끌고 있다. 이 코드는 오토배틀러의 전투 장면에서 영감을 받아 완벽하게 시뮬레이션 하지는 않지만, 오토배틀러에 대한 시각화를 구현하려 하였다.

(1) 주요 옵션 정리

옵션	정의	예시
runif()	지정된 범위 내에서 균등 분포를 따르는 난수를 생성	runif(n, min, max)
legend()	플롯에 범례를 추가하는데 사용	legend("topleft", legend = c("남자", "여자"))

beep()	beepr 패키지의 소리를 출력하는 함수	beep(sound = 3)
--------	------------------------	-----------------

(2) BombExplosion 함수 정의

```
BombExplosion <- function(center_x, center_y, radius = 0.2,
                          times = 3,
                          num_points = 100,
                          steps = 8,
                          sleep = 0.07) {
  for (rep_i in seq_len(times)) {
    center_ab <- cbind(runif(1, center_x - 0.03, center_x + 0.03),
                      runif(1, center_y - 0.03, center_y + 0.03))
    fire_points <- Unif_on_Circle(center_ab[1, 1], center_ab[1, 2], radius, num_points)

    for (step_i in seq_len(steps)) {
      expand_ratio <- step_i / steps
      for (j in seq_len(num_points)) {
        x_new <- center_ab[1, 1] + expand_ratio * (fire_points[j, 1] - center_ab[1, 1])
        y_new <- center_ab[1, 2] + expand_ratio * (fire_points[j, 2] - center_ab[1, 2])

        # 선 그리기 (빨간색)
        lines(x = c(center_ab[1, 1], x_new),
              y = c(center_ab[1, 2], y_new),
              col = rgb(1, 0, 0, 0.6), lwd = 2)

        if (step_i == steps) {
          points(x_new, y_new, pch = 16, col = rgb(1,0,0,0.8), cex = 0.9)
        }
      }
      Sys.sleep(sleep)
    }
  }
}
```

BombExplosion은 Fireworks에서 이펙트를 차용해 패배시 연출을 표현하게 한 함수이다. Unif_on_Circle를 똑같이 사용하였고, 작동원리도 Fireworks와 같다. 하지만 Fireworks와 다르게 패배 연출에 사용되었기 때문에 색도 빨간색으로 고정하였으며, 추후 설명할 플레이어 Agent 주변에 Fireworks가 표현되도록 설정하였다.

(3) generate_circle 함수 정의

```
generate_circle <- function(type) {
  list(
    x = runif(1, -game_size + circle_size, game_size - circle_size),
    y = runif(1, -game_size + circle_size, game_size - circle_size),
    type = type,          # "friend" 또는 "enemy"
    speed = move_range    # 초기 속도
  )
}
```

generate_circle() 함수는 아군("friend") 또는 적군("enemy") 동그라미 하나에 대한 정보(좌표, 타입, 속도)를 생성한다.

```
circles <- c(
  replicate(n_friends, generate_circle("friend"), simplify = FALSE),
  replicate(n_enemies, generate_circle("enemy"), simplify = FALSE)
)
```

n_friends개 아군, n_enemies개 적군을 각각 생성한 뒤, 하나의 리스트 circles로 합친다. 이후 코드는 circles 인자를 기반으로 설계되었다.

(4) move_circle 함수 정의

```
move_circle <- function(circle) {
  if (circle$type == "enemy") {
    circle$speed <- circle$speed + enemy_speed_increase
  } else {
    circle$speed <- circle$speed + speed_increase
  }

  # 이동
  new_x <- circle$x + runif(1, -circle$speed, circle$speed)
  new_y <- circle$y + runif(1, -circle$speed, circle$speed)

  circle$x <- max(-game_size + circle_size, min(game_size - circle_size, new_x))
  circle$y <- max(-game_size + circle_size, min(game_size - circle_size, new_y))

  circle
}
```

아군은 speed_increase만큼, 적군은 enemy_speed_increase만큼 속도가 커진다. 게임이 진행될수록 동그라미의 이동 범위가 넓어지고, 적군이 조금 더 빠른 속도로 커진다. 결과적으로 게임이 끝날쯤에는 적군이 더 강하게 움직인다는 것을 표현하고자 하였다. 동그라미의 이동은 x와 y의 좌표에 [-speed, speed] 범위 사이의 난수를 runif를 이용하여 새로운 위치를 정하게 하였다. 또한 동그라미가 게임판 바깥으로 나가지 않게 max함수를 이용하여 범위를 지정하였다.

(5) detect_collisions 동그라미간 충돌 감지 함수 정의

```
detect_collisions <- function(circles) {
  remaining_circles <- circles
  to_remove <- c()

  for (i in seq_along(circles)) {
    for (j in seq_along(circles)) {
      if (i < j) {
        dist_ij <- sqrt((circles[[i]]$x - circles[[j]]$x)^2 +
          (circles[[i]]$y - circles[[j]]$y)^2)
        if (!is.na(dist_ij) && dist_ij < 1.5 * circle_size) {

          # 충돌 시 사운드/메시지
        }
      }
    }
  }
}
```

```

        if (circles[[i]]$type == "friend" && circles[[j]]$type == "friend") {
          cat("사격금지! 아군이다!\n")
          beep(sound = 7)
        } else if (circles[[i]]$type == "enemy" && circles[[j]]$type == "enemy") {
          cat("뭐하는거야!\n")
          beep(sound = 11)
        } else {
          # 아군 vs 적군
          cat("으악!\n")
          beep(sound = 2)
        }
        to_remove <- c(to_remove, i, j)
      }
    }
  }
}

if (length(to_remove) > 0) {
  remaining_circles <- remaining_circles[-unique(to_remove)]
}
remaining_circles
}

```

두 동그라미가 서로 충돌하는지 여부를 판단할 때, 위 함수는 두 동그라미의 중심 거리(dist_ij)가 $1.5 * \text{circle_size}$ 보다 작으면 충돌로 간주하도록 설정한다. 만약 게임 중 충돌이 발생한다면 위 함수는 다음과 같은 메시지를 출력하며 동그라미들을 제거한다. 아군끼리 충돌했을 경우 “사격금지! 아군이다!”, 적군끼리 충돌했을 경우 “뭐하는거야!”, 아군과 적군이 충돌했을 경우 “으악!” 등 상황에 맞는 멘트를 띄운다. 그리고 beep() 함수를 통해 각 상황에 대응하는 소리를 출력한다. 이를 통해 플레이어는 청각적 피드백도 얻게 된다. 마지막으로 충돌한 두 동그라미의 인덱스를 to_remove 벡터에 기록해두었다가, 최종적으로 리스트에서 해당 인덱스를 삭제한다. 즉, 두 동그라미 모두 게임판에서 사라져 더 이상 그려지지 않는다.

(6) 게임판 생성

```

plot_game <- function() {
  par(bg = "black")

  # 게임판 크기 설정
  plot(0, 0,
       xlim = c(-game_size * 1.5, game_size * 1.5),
       ylim = c(-game_size * 1.5, game_size * 1.5),
       type = "n", bty = "n", axes = FALSE, xlab = "", ylab = "")

  # 게임판 테두리(흰색 사각형)
  lines(x = c(-game_size, game_size, game_size, -game_size, -game_size),
        y = c(-game_size, -game_size, game_size, game_size, -game_size),
        col = "white", lwd = 2)

  # 게임 제목
  title("전략적 우왕좌왕 전투", col.main = "white")
}

```

```

rect_x_left <- -game_size * 1.4
rect_x_right <- -game_size * 1.2
rect_y_bottom <- -0.5
rect_y_top <- 0.5

rect(rect_x_left, rect_y_bottom, rect_x_right, rect_y_top,
     col = "green", border = "white", lwd = 3)
text(x = (rect_x_left + rect_x_right)/2, y = 0,
     labels = "나", col = "white")

rect_x_left2 <- game_size * 1.2
rect_x_right2 <- game_size * 1.4
rect_y_bottom2 <- -0.5
rect_y_top2 <- 0.5

rect(rect_x_left2, rect_y_bottom2, rect_x_right2, rect_y_top2,
     col = "red", border = "white", lwd = 3)
text(x = (rect_x_left2 + rect_x_right2)/2, y = 0,
     labels = "상대", col = "white")

for (circle in circles) {
  points(circle$x, circle$y,
         col = ifelse(circle$type == "friend", "green", "red"),
         pch = 16, cex = 1.5)
}

n_friends_remaining <- sum(sapply(circles, function(c) c$type == "friend"))
n_enemies_remaining <- sum(sapply(circles, function(c) c$type == "enemy"))

legend("topright",
      legend = c(paste("아군:", n_friends_remaining),
                 paste("적군:", n_enemies_remaining)),
      col = c("green", "red"), pch = 16, bg = "black", text.col = "white")
}

```

Plot을 생성할 때 Fireworks의 기본 설정처럼 배경을 검은색으로 지정한다. 또한 게임판보다는 크게 $\pm(\text{game_size} \times 1.5)$, 즉 $[-3,3] \times [-3,3]$ 정도로 설정하여 넓은 범위를 확보한다. 그리고 plot 중앙에는 실제 게임판을 게임 판($[-2,2] \times [-2,2]$)을 시각화 한다. 플롯의 왼쪽 영역(-2.8 ~ -2.4 근처)에는 녹색 사각형을 그려 "나(아군) 진영"이라고 표시하고, 오른쪽 영역(2.4 ~ 2.8 근처)에는 빨간 사각형을 그려 "상대(적군) 진영"이라고 표기한다. 플롯의 우측 상단에는 범례(legend())를 생성하여 남은 아군과 남은 적군의 수를 텍스트로 표시하고, 이를 실시간으로 갱신하게 하였다.

(7) game_loop 게임 실행 함수 정의

```

game_loop <- function() {
  for (t in seq_len(game_time)) {
    Sys.sleep(1) # 1 초마다 업데이트

    # (전역) circles 갱신
  }
}

```

```

circles <- lapply(circles, move_circle)
circles <- detect_collisions(circles)

# 게임판 업데이트
plot_game()
}

# 최종 결과 판정
n_friends_remaining <- sum(sapply(circles, function(c) c$type == "friend"))
n_enemies_remaining <- sum(sapply(circles, function(c) c$type == "enemy"))

if (n_friends_remaining > n_enemies_remaining) {
  # 아군 승리
  mtext(paste("게임 종료! 아군 승리! 남은 아군:", n_friends_remaining),
        side = 1, line = 3, col = "green")
  title("축하합니다! 승리했습니다!", line = -2, col.main = "green")
  beep(sound = 3) # 승리 사운드

  # 폭죽 애니메이션 (sleep = 0.02 로 빠르게)
  Fireworks(num_fire = 8,
            num_points = 300,
            steps = 15,
            sleep = 0.01, # 더 빠르게
            color_alpha = 0.6,
            x_min = -game_size*1.2,
            x_max = game_size*1.2,
            y_min = -game_size*1.2,
            y_max = game_size*1.2)

} else if (n_friends_remaining < n_enemies_remaining) {
  # 적군 승리 -> 아군 패배
  mtext(paste("게임 종료! 적군 승리! 남은 적군:", n_enemies_remaining),
        side = 1, line = 3, col = "red")
  title("아쉽습니다. 패배하셨습니다!", line = -2, col.main = "red")

  # 아군 플레이어 쪽에 폭탄 연속 폭발 (sleep = 0.01 로 매우 빠르게)
  bomb_center_x <- -game_size * 1.3
  bomb_center_y <- 0
  BombExplosion(center_x = bomb_center_x,
               center_y = bomb_center_y,
               radius = 0.3,
               times = 4, # 연속 4 번 폭발
               num_points = 120,
               steps = 8,
               sleep = 0.01) # 매우 빠르게

} else {
  # 무승부
  mtext(paste("게임 종료! 무승부! 남은 아군:",
              n_friends_remaining, "남은 적군:", n_enemies_remaining),
        side = 1, line = 3, col = "yellow")
  title("무승부입니다!", line = -2, col.main = "yellow")
}
}

```

위 코드는 게임을 동작하게 하는 코드로 반복문은 총 game_time번 반복 수행되며, 매 반복마다

1초(Sys.sleep(1))씩 대기한다. 각 반복에서 동그라미들의 위치 및 속도를 업데이트하고 서로 부딪힌 동그라미가 있으면 제거하고, beep()에서 지정한 사운드를 낸다. 그리고 현재 남아 있는 동그라미들의 상태를 업데이트한다.

만약 지정된 게임 시간이 끝났다면 남아 있는 아군 수(n_friends_remaining)와 적군 수(n_enemies_remaining)를 세어 최종 승패를 결정한다. 아군이 적군보다 많으면 "축하합니다! 승리했습니다!"라는 메시지를 출력하고 Fireworks() 함수를 호출해 축하 폭죽 시뮬레이션을 실행한다. 또한 승리시에도 beep()함수를 적용하여 축하 팡파레 BGM을 재생한다. 그러나 적군이 아군보다 많으면 "아쉽습니다. 패배하셨습니다!"라는 메시지를 출력하고, 플레이어에게 BombExplosion() 함수를 호출해 빨간 폭탄 폭발 애니메이션을 실행한다. 마지막으로 아군 수와 적군 수가 같으면 특별한 연출은 하지 않고 "무승부" 메시지만 표시한다.